

Parte 2. Comprensión del flujo colaborativo

Desarrollo de Aplicaciones y Servicios Virtuales - Primer Examen Parcial

1. Flujo colaborativo

Planteamiento: Supón que quieres colaborar con el repositorio de otro desarrollador. Ordena y explica los siguientes elementos: Push, Fork, Pull Request, Clone, Merge, Commit, Review, Branch y Modificar archivos. Agrega cualquier operación que consideres necesaria.

Respuesta - Orden correcto:

1. **Fork** - Crear una copia del repositorio original dentro de mi propia cuenta de GitHub.
2. **Clone** - Descargar esa copia (mi fork) a mi computadora para trabajar localmente.
3. **Branch** - Crear una rama nueva dentro de mi repositorio local para aislar el trabajo que voy a hacer, sin afectar la rama principal.
4. **Modificar archivos** - Realizar los cambios de código o contenido dentro de esa rama.
5. **Commit** - Registrar esos cambios en el historial local de Git, con un mensaje que describa lo realizado.
6. **Push** - Enviar la rama con los commits desde mi repositorio local hacia mi fork en GitHub.
7. **Pull Request** - Solicitar formalmente al propietario del repositorio original que revise e incorpore mis cambios.
8. **Review** - El propietario (o el equipo) revisa el código propuesto, deja comentarios y decide si se aprueba o se piden ajustes.
9. **Merge** - Una vez aprobado, el propietario combina (fusiona) los cambios de mi rama dentro de la rama principal del repositorio original.

Operación adicional: Sync Fork - después de que el Pull Request fue aceptado (o si el repositorio original avanzó mientras yo trabajaba), debo sincronizar mi fork con el repositorio original para mantenerlo actualizado antes de continuar colaborando.

Explicación: Este flujo separa claramente el trabajo de un colaborador externo del repositorio oficial: el Fork y el Clone crean copias propias en las que se puede experimentar sin riesgo; el Branch aísla cada tarea; el Commit y el Push registran y publican el progreso; y el Pull Request, junto con el Review y el Merge, son el mecanismo de control que permite al dueño del proyecto revisar la calidad del código antes de integrarlo. Ningún cambio llega al repositorio original de forma automática: siempre pasa por una revisión humana.

2. Fork y Clone

Planteamiento: Analiza la afirmación: "Clone crea una copia del proyecto dentro de mi cuenta de GitHub". Indica si es correcta y explica la diferencia entre Fork y Clone.

Respuesta: La afirmación es **incorrecta**.

Explicación: Clone no crea nada dentro de GitHub; **descarga** una copia completa del repositorio (incluyendo su historial de commits) desde GitHub hacia la computadora del usuario, para poder trabajar con Git de forma local. Quien crea una copia del proyecto **dentro de una cuenta de GitHub** es el **Fork**: genera un repositorio independiente, propiedad de mi cuenta, que vive en los servidores de GitHub y que mantiene un vínculo de referencia con el repositorio original. En resumen:

- **Fork** = copia remota del repositorio, alojada en GitHub, bajo mi cuenta.
- **Clone** = copia local del repositorio (el mío, el de otra persona o un fork), descargada a mi computadora.

Es común usar ambas en conjunto: primero se hace Fork (para tener un repositorio propio en GitHub) y después Clone de ese fork (para poder trabajar en la computadora local).

3. Pull Request

Planteamiento: Supón que realizaste Fork, Clone, Branch, Modificar, Commit y Push. Responde: ¿los cambios ya forman parte del repositorio original? ¿Qué debe ocurrir para incorporarlos?

Respuesta: No, los cambios **todavía no** forman parte del repositorio original. Únicamente existen en la rama de mi **fork** (mi copia en GitHub), a donde llegaron gracias al Push. Para que se incorporen al repositorio original debo **abrir un Pull Request**, el propietario debe **revisarlo (Review)** y, si lo aprueba, realizar el **Merge**.

Explicación: El Push solo actualiza el repositorio remoto al que tengo permisos de escritura, que en este flujo es mi propio fork, no el repositorio original del otro desarrollador. El Pull Request es la solicitud formal que le indica al dueño del

repositorio original: “tengo cambios en mi fork, ¿los aceptas en tu proyecto?”. Hasta que el propietario apruebe y ejecute el Merge, el código sigue existiendo únicamente en mi copia; el repositorio original permanece sin cambios.

4. Request Changes

Planteamiento: El propietario revisa tu Pull Request y selecciona Request Changes. Explica qué debes hacer, si necesitas crear otro Pull Request y qué ocurre cuando realizas nuevamente push.

Respuesta: Debo corregir el código en la **misma rama** que usé para el Pull Request: modificar los archivos según las observaciones, hacer **commit** de esas correcciones y **push** a la misma rama de mi fork. **No es necesario crear un nuevo Pull Request**; el mismo Pull Request ya abierto se actualiza automáticamente.

Explicación: Un Pull Request está vinculado a una rama específica, no a un conjunto fijo de commits. Cuando hago push de nuevos commits a esa misma rama, GitHub los agrega automáticamente al Pull Request existente y notifica al revisor de que hay cambios nuevos que puede volver a evaluar. Esto evita duplicar solicitudes y mantiene toda la conversación y el historial de revisión en un solo lugar, hasta que finalmente el propietario apruebe y haga Merge.

5. Merge y repositorio local

Planteamiento: Un Pull Request fue aceptado y se realizó Merge en GitHub. Sin embargo, el repositorio local del propietario no contiene los cambios. Explica por qué sucede y qué operación debe realizarse.

Respuesta: Sucede porque el Merge se ejecutó **en el repositorio remoto (GitHub)**, y los repositorios remoto y local son copias independientes que no se sincronizan automáticamente. Para que el propietario tenga los cambios en su computadora, debe ejecutar `git pull` (o `git fetch` seguido de `git merge`) sobre su repositorio local.

Explicación: Git no mantiene el repositorio local y el remoto conectados en tiempo real; cada uno se actualiza solo cuando se ejecuta explícitamente una operación de red (push, pull o fetch). El Merge realizado en GitHub actualiza únicamente la rama principal alojada en el servidor. El propietario, al trabajar día a día desde su copia local, necesita traer manualmente esos cambios con `git pull` para que su carpeta de trabajo y su historial local queden al día con lo que ya existe en GitHub.

6. Sync Fork

Planteamiento: Tu Fork fue creado varios días atrás y el repositorio original recibió nuevos commits. Explica qué herramienta utilizarías, qué repositorio se actualiza y qué diferencia existe entre Sync Fork y git pull.

Respuesta: Utilizaría la opción **Sync Fork** de GitHub. Esta operación actualiza **mi fork (el repositorio remoto en mi cuenta de GitHub)**, trayendo hacia él los commits nuevos que existen en el repositorio original (upstream).

Explicación: Sync Fork y `git pull` resuelven problemas distintos aunque parezcan similares. `git pull` actualiza mi **repositorio local** trayendo los cambios desde un repositorio remoto (por defecto, mi propio fork en GitHub). **Sync Fork**, en cambio, actualiza mi **fork remoto** trayendo los cambios desde el repositorio **original** (el que no es mío). Es decir, Sync Fork trabaja entre dos repositorios remotos (original -> mi fork), mientras que `git pull` trabaja entre un repositorio remoto y mi máquina local (remoto -> local). Después de usar Sync Fork normalmente sigue siendo necesario un `git pull` para que esos cambios también lleguen a mi copia local.